

Parameter-Free Universal Trust-Region Methods for Nonconvex–Strongly-Concave Minimax Optimization

1 Introduction

We consider smooth nonconvex–strongly-concave (NC–SC) minimax optimization

$$\min_{x \in \mathbb{R}^{d_x}} \max_{y \in \mathbb{R}^{d_y}} f(x, y), \quad (1.1)$$

where $f(x, \cdot)$ is strongly concave for every x , while $f(\cdot, y)$ may be nonconvex. Strong concavity makes the inner maximizer $y^*(x) := \arg \max_y f(x, y)$ well defined and gives rise to the primal function $P(x) := \max_y f(x, y)$. Under standard smoothness assumptions, P is twice differentiable but generally nonconvex. An approximate first-order stationary point of P therefore need not capture local optimality, which motivates the search for approximate second-order stationary points.

Second-order algorithms for NC–SC minimax optimization have been developed using cubic regularization, trust-region methods, and other Newton-type updates [1, 2, 3, 4, 5]. A representative method is Minimax Cubic Newton (MCN) [1], which approximates the gradient and Hessian of P through inner maximization and achieves the characteristic $O(\epsilon^{-3/2})$ second-order oracle complexity. Several subsequent methods attain the same $\epsilon^{-3/2}$ -type outer dependence under related second-order stationarity criteria. More recently, joint-minimization reformulations have led to methods that avoid repeatedly solving the inner maximization [6, 7].

For methods that construct approximate derivatives of P through inner maximization, the outer iteration complexity alone does not describe the total computational work. At each outer iterate x_k , one must approximate $y^*(x_k)$ accurately enough that the resulting first- and second-order information of P can be used reliably. In MCN, accelerated gradient ascent is used for this purpose. Its first-order oracle complexity is stated in $\tilde{O}(\epsilon^{-3/2})$ form, and the explicit bound contains a logarithmic dependence on the target accuracy multiplying the leading outer-order term. This raises a natural question: can one retain the $O(\epsilon^{-3/2})$ second-order iteration dependence while removing the multiplicative logarithmic factor from the total inner-gradient complexity?

We answer this question affirmatively by combining existing primal-function estimators with an analysis of the accumulated inner work. The implicit gradient estimator studied by Ablin et al. [8] has known quadratic accuracy with respect to the inner solution error; see their Proposition 2.3. The same total-gradient and Schur-complement expressions also appear in Newton-type minimax methods [9]. We use these quantities, together with a Newton correction of the value, as oracle ingredients and derive the explicit error bounds needed under our NC–SC assumptions. The complexity improvement comes from combining these bounds with warm starts and a cumulative cubic bound on the outer steps. This path estimate controls the total cost of restoring inner accuracy across successive maximization problems.

Our method builds on the universal trust-region (UTR) framework of Jiang et al. [10]. We first formulate a general inexact UTR interface in which working queries provide approximate values, gradients, and Hessians,

and validation queries check possible solutions. Its convergence theorem controls both the outer iteration count and the cumulative path needed for inner-work analysis. The minimax algorithm realizes this interface through corrected primal-function oracles and persistent calls to SCAR [11]. It retains the inner solver’s curvature estimates across successive maximization problems and adjusts inner accuracy to the current outer scale. This gives the desired leading outer and inner complexity without supplying smoothness or strong-concavity parameters to either component.

1.1 Main Contributions

Our contributions concern the total inner work and the removal of problem parameters from the executable minimax method. The existing implicit gradient and Schur-complement Hessian are ingredients of the construction.

Sharper total inner-gradient complexity. Let ℓ and ρ denote the Lipschitz constants of the joint gradient and Hessian of f , respectively, let $\mu > 0$ be the strong-concavity modulus of $f(x, \cdot)$, and set

$$\kappa := \ell/\mu, \quad \Delta_P := P(x_0) - \inf_x P(x), \quad M_P := 4\sqrt{2}\kappa^3\rho.$$

Algorithm 2 combines the implicit gradient and Schur-complement Hessian with an adaptive UTR step and accesses inner maximization through observable gradient residuals. The key estimate is a cumulative cubic bound on the accepted outer steps, which amortizes the logarithmic costs of successive warm-started inner calls. Persistent SCAR also shares its curvature calibration across the whole run. For each fixed regularity class and sufficiently small ϵ , **Theorem 3.1** gives

$$\begin{aligned} Q &= O\left(\Delta_P \kappa^{3/2} \sqrt{\rho} \epsilon^{-3/2}\right) + O_{\text{fixed}}(\log^3(e/\epsilon)), \\ N_y &= O\left(\Delta_P \kappa^2 \sqrt{\rho} \epsilon^{-3/2}\right) + O_{\text{fixed}}(\sqrt{\kappa} \log^3(e/\epsilon)), \end{aligned}$$

where Q counts trust-region subproblems and N_y counts all inner-gradient evaluations. Here O_{fixed} permits constants depending on the fixed payoff, regularity constants, and initialization, but not on ϵ . Neither $\log(1/\epsilon)$ nor $\log \kappa$ multiplies the displayed leading inner term. In particular, this removes the leading target-accuracy logarithm in the explicit inner-gradient bound of MCN. As a technical tool, **Theorem 2.1** gives a general inexact UTR complexity and path estimate; the minimax analysis invokes it after verifying the corrected-oracle error bounds.

A fully parameter-free second-order method. **Algorithm 2** uses only the initialization, target tolerance, and universal numerical constants. Neither the outer scheme nor SCAR requires ℓ , μ , ρ , κ , or M_P as an input. Working inner accuracy is restored through a number of residual halvings determined by the observed outer step and current scale; more accurate validation is reserved for candidate solutions. The output satisfies

$$\|\nabla P(x_{\text{out}})\| < \epsilon, \quad \lambda_{\min}(\nabla^2 P(x_{\text{out}})) \geq -\frac{85}{16\sqrt{6}}\sqrt{M_P\epsilon}.$$

A universal constant rescaling of the internal tolerance gives the standard ϵ -second-order guarantee defined in **Definition 2.1**. The guarantee is class-uniform asymptotic: after fixing common (ℓ, μ, ρ) , one threshold $\epsilon_0(\ell, \mu, \rho) > 0$ works for every admissible payoff and initialization. The algorithm does not know this threshold or certify that the requested accuracy lies below it. **Section 5** explains this qualification through the residual-only obstruction and the role of validation.

Table 1 compares methods whose stated second-order guarantees and oracle counts match the quantities considered here. Cubic-LocalMinimax [2], whose complexity is stated using a different local-minimax

Table 1: Complexity comparison of deterministic second-order methods for smooth NC–SC minimax optimization. Initial optimality gaps and universal numerical constants are suppressed. For cited methods, regularity-parameter dependence is shown when reported in a directly comparable form; entries without such factors report only dependence on ϵ . For our method the entries are leading terms, valid in the class-uniform small-accuracy regime of [Theorem 3.1](#).

Method	Outer complexity	Total inner-gradient evaluations	Parameter-free
MCN [1]	$O(\kappa^{3/2} \sqrt{\rho} \epsilon^{-3/2})$	$\tilde{O}(\kappa^2 \sqrt{\rho} \epsilon^{-3/2})$	No
MINIMAX-TR / MINIMAX-TRACE [3]	$O(\epsilon^{-3/2})$	Not reported	No
GRTR / LMNegCur [4]	$\tilde{O}(\kappa^{3/2} \sqrt{\rho} \epsilon^{-3/2})$	$\tilde{O}(\kappa^2 \sqrt{\rho} \epsilon^{-3/2})$	No
HSDA / IHSDA [5]	$\tilde{O}(\epsilon^{-3/2})$	Not reported	No
Algorithm 2 (ours)	$O(\kappa^{3/2} \sqrt{\rho} \epsilon^{-3/2})$	$O(\kappa^2 \sqrt{\rho} \epsilon^{-3/2})$	Yes

Here $\kappa = \ell/\mu$ and ρ is the Hessian-Lipschitz constant of f . The notation $\tilde{O}(\cdot)$ is retained when used in the cited reference. *Not reported* means that the cited work does not state a separate comparable total inner-gradient bound. The complete bounds for [Algorithm 2](#) include additive $O_{\text{fixed}}(\log^3(e/\epsilon))$ and $O_{\text{fixed}}(\sqrt{\kappa} \log^3(e/\epsilon))$ terms for the outer and inner counts, respectively. Its executable steps use no problem parameters; the accuracy threshold in its guarantee is unknown.

stationarity measure, is discussed below without reparameterizing it for the table. The recent single-loop regularized Newton method of Ma et al. [7] has no inner maximization loop of the form measured by the table’s third column.

1.2 Related Work

Implicit differentiation and primal-function estimators. Ablin et al. [8] analyze the implicit gradient estimator for functions defined through optimization and establish its quadratic error in Proposition 2.3. In minimax optimization, Zhang et al. [9] use the corresponding total gradient and Schur-complement Hessian; see their equations (1.1)–(1.2). These estimators are ingredients of the present work. Our contributions concern their integration into an inexact UTR method, the accumulated inner-gradient complexity, and parameter-free tracking across changing inner objectives.

First-order methods for NC–SC minimax optimization. First-order methods for NC–SC minimax problems have been studied extensively. Representative approaches include gradient descent-ascent, multi-step ascent, proximal schemes, and accelerated methods [12, 13, 14]. Their complexity for finding first-order stationary points of the primal function has also been studied from both upper- and lower-bound perspectives; see, for example, Zhang et al. [15] and the references therein. These results form the first-order counterpart to the second-order stationarity problem considered in this paper.

Second-order methods for NC–SC minimax optimization. Luo et al. [1] proposed MCN, which alternates accelerated gradient ascent for approximating $y^*(x)$ with a cubic-regularized update based on approximate first- and second-order information of the primal function. Its leading bounds are $O(\Delta_P \kappa^{3/2} \sqrt{\rho} \epsilon^{-3/2})$ second-order oracle evaluations and $\tilde{O}(\Delta_P \kappa^2 \sqrt{\rho} \epsilon^{-3/2})$ first-order oracle evaluations. Chen et al. [2] proposed Cubic-LocalMinimax and developed inexact and stochastic variants, using cubic regularization to

obtain global convergence toward local minimax points. Their complexity is formulated using a different stationarity measure, so we do not convert it to the accuracy parameterization used in [Table 1](#).

Trust-region approaches were developed by Yao and Xu [\[3\]](#), who proposed MINIMAX-TR and MINIMAX-TRACE. Wang and Xu [\[4\]](#) subsequently proposed gradient-norm regularized trust-region and Levenberg–Marquardt methods, together with inexact variants, and established $\tilde{O}(\kappa^{3/2}\sqrt{\rho}\epsilon^{-3/2})$ outer complexity and $\tilde{O}(\kappa^2\sqrt{\rho}\epsilon^{-3/2})$ total gradient-ascent complexity, suppressing the initial-gap dependence. More recently, Chen et al. [\[5\]](#) proposed homogeneous second-order descent-ascent methods and developed an inexact Lanczos implementation for large-scale problems.

A recent alternative is to avoid repeated inner maximization altogether. Ma et al. [\[6\]](#) introduced an equivalent joint-minimization reformulation that preserves first- and second-order stationarity and used it to develop a parameter-free line-search framework. Building on this reformulation, Ma et al. [\[7\]](#) proposed a single-loop regularized Newton method with $O(\epsilon^{-3/2})$ global iteration complexity and local superlinear convergence. This line of work addresses the cost of the inner loop by changing the formulation, whereas the present paper studies total inner-gradient complexity when approximate derivatives of the primal function are constructed through inner maximization.

Adaptive and parameter-free optimization. Adaptive optimization methods aim to reduce or eliminate prior knowledge of problem regularity constants. In smooth nonconvex optimization, the universal trust-region framework [\[10\]](#) uses gradient-scaled trust-region updates and provides the outer mechanism used here. On the inner side, Lan et al. [\[11\]](#) developed SCAR, a parameter-free accelerated method for smooth strongly convex optimization that adapts to unknown smoothness and strong-convexity parameters. The general inexact UTR theorem and the SCAR guarantees are presented as tools in [Section 2](#); their supporting proofs are deferred to the appendices.

Related adaptive and parameter-free methods have also been developed for minimax optimization and monotone variational inequalities [\[16, 17\]](#). More recently, Wang et al. [\[18\]](#) proposed a fully parameter-free second-order method for convex–concave minimax optimization, requiring neither the Hessian-Lipschitz constant nor an a priori distance bound to a solution. For NC–SC minimax problems, Ma et al. [\[6\]](#) developed a parameter-free line-search framework based on a joint-minimization reformulation. Our method retains the primal-function formulation and combines a gradient-scaled UTR step with persistent SCAR tracking across successive inner maximization problems.

1.3 Organization

[Section 2](#) introduces the inexact UTR and SCAR tools and the NC–SC setting. [Section 3](#) presents the corrected oracles, their SCAR realization, and the minimax complexity theorem. [Section 4](#) proves that the minimax oracles satisfy the UTR interface and bounds their total inner work. [Section 5](#) discusses the scope of the guarantee, and [Section 6](#) concludes the paper. Only the proofs for the general UTR and SCAR tools are deferred to [Appendices A](#) and [B](#).

2 Notations and preliminaries

The norm $\|\cdot\|$ is Euclidean for vectors and spectral for matrices. For a symmetric matrix H , $\lambda_{\min}(H)$ denotes its smallest eigenvalue. We write $\log_+(t) = \max\{\log t, 0\}$ for $t > 0$ and set $\log_+(0) = 0$. Unsubscripted logarithms are natural. Universal $O(\cdot)$ constants do not depend on problem parameters. For $b_\epsilon \geq 0$, the relation $a_\epsilon = O_{\text{fixed}}(b_\epsilon)$ as $\epsilon \downarrow 0$ means that, for every fixed choice of the payoff, regularity constants, and initialization, there exist $C > 0$ and $\epsilon' > 0$ such that $|a_\epsilon| \leq Cb_\epsilon$ for all $0 < \epsilon \leq \epsilon'$. These constants need not be uniform over problem instances. The notation $\tilde{O}(\cdot)$ suppresses logarithmic factors.

Assumption 2.1 (Hessian regularity). *The function F is twice continuously differentiable and, for some $M > 0$,*

$$\|\nabla^2 F(x) - \nabla^2 F(z)\| \leq M\|x - z\| \quad \forall x, z \in \mathbb{R}^n. \quad (2.1)$$

Lemma 2.1 (Second-order Taylor bounds). *If $\nabla^2 F$ is M -Lipschitz, then, for all x, d ,*

$$\|\nabla F(x + d) - \nabla F(x) - \nabla^2 F(x)d\| \leq \frac{M}{2}\|d\|^2. \quad (2.2a)$$

$$F(x + d) \leq F(x) + \nabla F(x)^T d + \frac{1}{2}d^T \nabla^2 F(x)d + \frac{M}{6}\|d\|^3. \quad (2.2b)$$

These estimates are standard; see Nesterov [19, Lemma 1.2.4].

Definition 2.1 (Approximate second-order stationarity). *Let F be twice continuously differentiable with an M -Lipschitz Hessian, where $M > 0$. For $\epsilon > 0$, a point x is an ϵ -second-order stationary point (ϵ -SOSP) if*

$$\|\nabla F(x)\| \leq \epsilon, \quad \lambda_{\min}(\nabla^2 F(x)) \geq -\sqrt{M\epsilon}.$$

All trust-region subproblems are solved globally. For the quadratic model $g^T d + d^T B d / 2$ on $\|d\| \leq R$, a solution is accompanied by a multiplier $\lambda \geq 0$ satisfying

$$(B + \lambda I)d = -g, \quad B + \lambda I \succeq 0, \quad \|d\| \leq R, \quad \lambda(\|d\| - R) = 0. \quad (2.3)$$

These conditions are necessary and sufficient [20, Section 8.7]. We use exact arithmetic for these solves, eigenvalue tests, and applications of inverse Hessian blocks; the reported oracle counts do not include their arithmetic cost.

2.1 Parameter-free UTR with inexact oracles

We first isolate an outer method for minimizing a general objective F . It uses approximate values, gradients, and Hessians through a working oracle $\mathcal{W}$, and checks possible solutions through a validation oracle $\mathcal{V}$. The oracle calls may use auxiliary data, such as an inner iterate, and retain solver state between calls. Their accuracy requirements below are analytical assumptions, not error certificates that the algorithm is required to compute.

A working query returns a tuple $(\hat{F}, \hat{g}, \hat{H})$ at a point x and scale σ , with $\hat{H}$ symmetric. There are three types of query: initialization at x_0 ; a trial query at $x + d$, warm-started from the stored data at x ; and a refresh at the unchanged x after the scale is doubled. A validation query uses the working candidate's data to return (g_v, H_v) at that same candidate point. All calls must finish in finite time. Numerical tuples and their auxiliary point data are distinct from any shared solver calibration: rejecting a candidate discards the former but need not roll back the latter.

Assumption 2.2 (Inexact working and validation oracles). *Fix $\epsilon > 0$ and $\sigma_0 > 0$. For constants $C_g, C_v \geq 0$, every working tuple returned at a generated scale $\sigma \geq \sigma_0$ satisfies*

$$\|\hat{g} - \nabla F(x)\| \leq C_g \frac{\epsilon}{\sigma^2}, \quad |\hat{F} - F(x)| \leq C_v \frac{\epsilon^{3/2}}{\sigma^3}. \quad (2.4)$$

Whenever $\sigma^2 \geq M/6$, it additionally satisfies

$$\|\hat{g} - \nabla F(x)\| \leq \frac{\epsilon}{1024}, \quad \|\hat{H} - \nabla^2 F(x)\| \leq \frac{\sigma\sqrt{\epsilon}}{64}, \quad |\hat{F} - F(x)| \leq \frac{\epsilon^{3/2}}{4096\sigma}. \quad (2.5)$$

Every validation query, including those with $\sigma^2 < M/6$, returns a symmetric H_v and satisfies

$$\|g_v - \nabla F(x)\| \leq \frac{\epsilon}{1024}, \quad \|H_v - \nabla^2 F(x)\| \leq \frac{\sigma\sqrt{\epsilon}}{32}. \quad (2.6)$$

The constants M, C_g, C_v are not supplied to the outer algorithm. The requirements hold for every actual query, including repeated queries at the same point and queries after a scale change.

At a stored working tuple, set

$$h := \max\{\|\hat{g}\|, \epsilon\}, \quad R(h, \sigma) := \frac{\sqrt{h}}{4\sigma}, \quad (2.7)$$

and solve

$$\min_{\|d\| \leq R(h, \sigma)} \hat{g}^T d + \frac{1}{2} d^T \left(\hat{H} + \frac{\sigma\sqrt{\epsilon}}{64} I + \sigma\sqrt{h} I \right) d. \quad (2.8)$$

For its global solution and multiplier (d, λ) , define

$$r := \|d\|, \quad W := \sigma\sqrt{h} r^2, \quad U := \lambda r^2, \quad T := \frac{\epsilon^{3/2}}{\sigma}. \quad (2.9)$$

An ordinary trial is accepted when

$$\hat{F} - \hat{F}^+ \geq \frac{W}{8} + \frac{U}{4} - \frac{T}{1024}, \quad \|\hat{g}^+\| \leq \frac{h}{2} + \lambda r. \quad (2.10)$$

The exception is an interior step at $h = \epsilon$, which invokes validation rather than the ordinary acceptance test.

Remark 2.1 (Why working and validation oracles?). *The two roles separate the information needed to make progress from the information needed to return a solution. Working tuples build the model (2.8) and evaluate the acceptance tests (2.10). Their accuracy adapts to the current scale; the small-error bounds (2.5) are required only once $\sigma^2 \geq M/6$. Since M is unknown, an interior step at $h = \epsilon$ merely nominates a candidate: at an underestimated scale, the working derivatives need not justify termination.*

Validation checks that candidate with derivative-error bounds (2.6) that hold at every generated scale, including $\sigma^2 < M/6$. It needs no function value because it is used only to test stationarity. Thus the ordinary trials use working information, whereas only potential stopping points receive a validation query. The two oracles need not be different solvers: in the minimax construction, both use corrected estimates computed with SCAR, and validation refines the candidate's inner iterate. The stated error bounds are proved under the theorem's assumptions, not computed as certificates by the algorithm.

Algorithm 1: Parameter-free inexact adaptive UTR

Input: $x_0, \epsilon > 0, \sigma_0 > 0$, working oracle $\mathcal{W}$, and validation oracle $\mathcal{V}$

```

1  $x \leftarrow x_0, \sigma \leftarrow \sigma_0$ ; initialize and store a working tuple using  $\mathcal{W}$ 
2 while true do
3   Set  $(h, R)$  by (2.7), solve (2.8), and obtain  $(d, \lambda)$ 
4   Set  $x^+ \leftarrow x + d$ , form  $(r, W, U, T)$  by (2.9), and set accepted  $\leftarrow$  false
5   Obtain a candidate working tuple at  $x^+$  using a trial query of  $\mathcal{W}$  from the stored data at  $x$ 
6   if  $h = \epsilon$  and  $r < R(h, \sigma)$  then
7     Obtain  $(g_v, H_v)$  at  $x^+$  using  $\mathcal{V}$ 
8     if  $\|g_v\| \leq \epsilon/2$  and  $\lambda_{\min}(H_v) \geq -(21/8)\sigma\sqrt{\epsilon}$  then
9       return  $x^+$ 
10    end
11  else
12    if (2.10) holds then
13      accepted  $\leftarrow$  true
14    end
15  end
16  if accepted then
17    Replace  $x$  and the stored tuple and auxiliary data by the candidate; keep  $\sigma$  unchanged
18  else
19    Discard candidate and validation data; set  $\sigma \leftarrow 2\sigma$ 
20    Refresh the working tuple at the unchanged  $x$  using  $\mathcal{W}$  from the old stored auxiliary data
21  end
22 end

```

An accepted tuple is kept unchanged until the next accepted step or scale refresh. In particular, additional refinement of a stored oracle is not allowed without accounting for its change. A successful validation returns the candidate immediately. Any failed trial, including a failed validation, instead doubles the scale and refreshes the working tuple at the old stored point. The rejected candidate's point-dependent data are discarded; shared solver calibration may be retained. The scale is nondecreasing; this is essential for summing the changes of the inexact function-value proxy. The outer method uses no regularity or error constants. A fully parameter-free implementation also requires parameter-free oracle routines; Section 3 supplies them for minimax optimization.

Theorem 2.1 (Inexact UTR complexity and path budget). *Suppose F satisfies Assumption 2.1, with $F^* := \inf_x F(x) > -\infty$, and that the finite oracle routines satisfy Assumption 2.2 for the chosen $\epsilon > 0$ and $\sigma_0 > 0$. Run Algorithm 1 with globally solved subproblems. Write*

$$\Delta_F := F(x_0) - F^*, \quad h_0 := \max\{\|\hat{g}_0\|, \epsilon\}, \quad \bar{\sigma} := \max\{\sigma_0, 2\sqrt{M/6}\},$$

where $\hat{g}_0$ is the initialized working gradient, and define

$$B_\epsilon := \Delta_F + \frac{23C_v\epsilon^{3/2}}{7\sigma_0^3} + \frac{\epsilon^{3/2}}{512\sigma_0} \left[\left\lceil \log_2 \frac{h_0}{\epsilon} \right\rceil + 1 + \log_2 \left(1 + \frac{5C_g}{4\sigma_0^2} \right) \right]. \quad (2.11)$$

The algorithm terminates, every generated scale is at most $\bar{\sigma}$, and its output satisfies

$$\|\nabla F(x_{\text{out}})\| < \epsilon, \quad \lambda_{\min}(\nabla^2 F(x_{\text{out}})) \geq -\frac{85}{32}\bar{\sigma}\sqrt{\epsilon}. \quad (2.12)$$

If K is the number of accepted steps, J the number of failed trials, and Q the total number of subproblems, then

$$K \leq 1024\bar{\sigma}B_\epsilon\epsilon^{-3/2}, \quad J \leq \left\lceil \log_2 \max \left\{ 1, \frac{\sqrt{M/6}}{\sigma_0} \right\} \right\rceil, \quad Q = K + J + 1. \quad (2.13)$$

There are at most $J + 1$ validation calls. Moreover, every stored iterate and the accepted steps of every finite prefix satisfy

$$F(x) - F^* \leq B_\epsilon, \quad \sum_{\text{accepted}} \sigma_k^2 \|d_k\|^3 \leq 8B_\epsilon. \quad (2.14)$$

The bounds on K and J also apply to every finite prefix.

The theorem separates outer progress from the construction and cost of the oracles. Exact values and derivatives satisfy its hypotheses with $C_g = C_v = 0$. In that case the displayed budget gives

$$Q = O\left(\bar{\sigma}\Delta_F\epsilon^{-3/2} + \frac{\bar{\sigma}}{\sigma_0} \left[1 + \log_+ \frac{h_0}{\epsilon}\right] + \log_+ \frac{\sqrt{M}}{\sigma_0} + 1\right).$$

Thus the exact-oracle case remains included, with the dependence on the initial scale explicit. We use the same nondecreasing-scale algorithm for exact and inexact oracles, rather than require a separate exact-oracle result. The proof of the general theorem, including its path and stored-value bounds, is in [Appendix A](#).

Remark 2.2 (Sharper exact-oracle additive overhead). *The preceding display is the specialization of the unified inexact-oracle proof and deliberately retains the factors used to pay for possible value and gradient perturbations. If every working and validation call returns the exact function value, gradient, and Hessian, those perturbation payments disappear. A separate exact-oracle bookkeeping of the same scale updates then gives the sharper additive bound*

$$Q = O\left(\bar{\sigma}\Delta_F\epsilon^{-3/2} + \log_+ \frac{h_0}{\epsilon} + \log_+ \frac{\sqrt{M}}{\sigma_0} + 1\right). \quad (2.15)$$

Here $h_0 = \max\{\|\nabla F(x_0)\|, \epsilon\}$ and $\bar{\sigma} = \max\{\sigma_0, 2\sqrt{M/6}\}$. In particular, when $\sigma_0 = O(\sqrt{M})$, this becomes

$$Q = O\left(\Delta_F\sqrt{M}\epsilon^{-3/2} + \log_+ \frac{\|\nabla F(x_0)\|}{\epsilon} + 1\right),$$

so the search logarithms remain additive and do not multiply the leading $\epsilon^{-3/2}$ term. The sharper estimate is an exact-oracle specialization; the general inexact theorem above is the statement used by the NC-SC construction.

2.2 SCAR and persistent residual reduction

The inner maximization will use SCAR, the parameter-free strongly convex gradient-minimization method of Lan et al. [11]. Its computational building block is accumulative regularization (AR). We give the AR procedure and the residual-reduction loop explicitly, then distinguish the original SCAR guarantee from the persistent implementation used here. All oracle counts in this subsection count gradient evaluations; line searches also use function values.

The AR building block. The procedure below is Lan et al. [11, Algorithm 4.2], with their backtracking routine (Algorithm 4.1) written inline. We use α_s for its regularization to distinguish it from the outer UTR scale σ . Its accelerated subsolver $\mathcal{A}$ is the line-search implementation described after their Assumption 4.1, treating the quadratic regularizer as a proximable term. For

$$\phi_s(w) := \phi(w) + \frac{\alpha_s}{2} \|w - \bar{u}_s\|^2, \quad u_s^* := \arg \min_w \phi_s(w),$$

the subsolver, started at u_{s-1} , supplies iterates u_s^k and computed estimates L_s^k satisfying

$$\phi_s(u_s^k) - \phi_s(u_s^*) \leq \frac{L_s^k}{k^2} \|u_{s-1} - u_s^*\|^2, \quad 0 < L_s^k \leq 4L. \quad (2.16)$$

It stops at the first $k \geq 8\sqrt{2L_s^k/\alpha_s}$. Only this computable stopping test is used; L and u_s^* occur in the guarantee, not in the input. This specifies the imported accelerated subsolver required by the AR theorem.

Procedure AR($\phi, u, \alpha, \mathcal{M}$)

Input: Objective ϕ , point u , regularization $\alpha > 0$, and line-search estimate $\mathcal{M} > 0$

Output: Point v and updated estimate $\mathcal{M}^+$

```

1 Set  $u_0 = \bar{u}_0 = u$ ,  $\alpha_0 = 0$ , and  $\mathcal{M}_0 = \mathcal{M}$ 
2 for  $s = 1, 2, \dots$  do
3   Set  $\alpha_s = 4^{s-1}\alpha$  and  $\bar{u}_s = (\alpha_{s-1}/\alpha_s)\bar{u}_{s-1} + (1 - \alpha_{s-1}/\alpha_s)u_{s-1}$ 
4   Obtain  $u_s$  by applying  $\mathcal{A}$  to  $\phi_s$  from  $u_{s-1}$ , with the stopping test following (2.16)
5   Set  $\mathcal{M}_s = \mathcal{M}_{s-1}/2$  and  $p_s = \nabla\phi_s(u_s)$ 
6   while true do
7     Set  $w_s = u_s - p_s/[2(\mathcal{M}_s + \alpha_s)]$ 
8     if  $\phi_s(w_s) - \phi_s(u_s) - p_s^T(w_s - u_s) \leq \frac{\mathcal{M}_s + \alpha_s}{2} \|w_s - u_s\|^2$  then
9       break
10    else
11      Set  $\mathcal{M}_s \leftarrow 2\mathcal{M}_s$ 
12    end
13  end
14  if  $\alpha_s \geq \mathcal{M}_s$  then
15    return  $(v, \mathcal{M}^+) = (u_s, \mathcal{M}_s)$ 
16  end
17 end
```

The backtracking point w_s is used only to test a local inequality; the AR output is u_s , not w_s . The returned $\mathcal{M}^+$ is not a certified global upper bound on L .

SCAR as repeated residual halving. The next two procedures write Lan et al. [11, Algorithm 5.1] as a halving call followed by a residual-target loop, with an initial residual check. The pair $(\nu, \mathcal{M})$ is passed by reference: every update is retained by the caller, even if an AR candidate fails the halving test.

Procedure SCARHalf($\phi, u; \nu, \mathcal{M}$)

Input: Objective ϕ , point u , and shared calibration $\nu, \mathcal{M} > 0$

Output: Point with residual at most $\|\nabla\phi(u)\|/2$; updated calibration

```
1 Compute and freeze  $R = \|\nabla\phi(u)\|$ 
2 if  $R = 0$  then
3   | return  $u$ 
4 end
5 while true do
6   | Run  $(v, \mathcal{M}^+) = \text{AR}(\phi, u, \nu/10, \mathcal{M})$  with freshly initialized accelerated momentum
7   | Set  $\mathcal{M} \leftarrow \mathcal{M}^+$ 
8   | if  $\|\nabla\phi(v)\| \leq R/2$  then
9     | return  $v$ 
10  | end
11  | Discard  $v$  and set  $\nu \leftarrow \nu/4$ ; keep  $u$  and  $R$  unchanged
12 end
```

Procedure SCARTarget($\phi, u, \tau; \nu, \mathcal{M}$)

Input: Objective ϕ , point u , target $\tau > 0$, and shared calibration $\nu, \mathcal{M} > 0$

Output: Point with residual at most τ ; updated calibration

```
1 while  $\|\nabla\phi(u)\| > \tau$  do
2   | Set  $u \leftarrow \text{SCARHalf}(\phi, u; \nu, \mathcal{M})$ 
3 end
4 return  $u$ 
```

For a single objective, initializing $(\nu, \mathcal{M})$ once and running SCARTARGET gives SCAR with the harmless initial stopping check. The following fact records the imported guarantees.

Fact 2.1 (SCAR and AR; Lan et al. [11, Theorems 4.1 and 5.1]). *Let $\phi : \mathbb{R}^m \rightarrow \mathbb{R}$ be differentiable and μ -strongly convex, with an L -Lipschitz gradient, where $0 < \mu \leq L$, and let $u^* = \arg \min_u \phi(u)$. For any u , $\nu > 0$, and $0 < \mathcal{M} \leq 4L$, the AR subroutine*

$$(v, \mathcal{M}^+) = \text{AR}(\phi, u, \nu/10, \mathcal{M})$$

returns

$$\|\nabla\phi(v)\| \leq \frac{\nu}{2} \|u - u^*\|, \quad 0 < \mathcal{M}^+ \leq 2L, \quad (2.17)$$

using $O(1 + \sqrt{L/\nu})$ gradient evaluations. For any residual target $\tau > 0$, SCAR initialized at u with $\nu_0 \geq \mu$ and $0 < \mathcal{M}_0 \leq 4L$ returns $\hat{u}$ satisfying

$$\|\nabla\phi(\hat{u})\| \leq \tau \quad (2.18)$$

within

$$O\left(\sqrt{\frac{L}{\mu}} \left[1 + \log \frac{\nu_0}{\mu} + \log_+ \frac{\|\nabla\phi(u)\|}{\tau}\right]\right) \quad (2.19)$$

gradient evaluations. If the initial residual already satisfies the target, SCAR returns the initial point. All constants hidden in these bounds are universal, and neither routine requires L or μ as an input. The AR bound applies also when $L = \mu$, so the SCAR conclusion extends to this endpoint without a parameter-dependent branch.

An admissible initialization is obtained from two gradient evaluations. For any $z \neq u$, define

$$a(\phi; u, z) := \frac{\|\nabla\phi(z) - \nabla\phi(u)\|}{\|z - u\|}. \quad (2.20)$$

Strong convexity and gradient Lipschitz continuity imply

$$\mu \leq a(\phi; u, z) \leq L. \quad (2.21)$$

Thus $\nu_0 = \mathcal{M}_0 = a(\phi; u, z)$ initializes SCAR without knowing either problem parameter. For the minimax problem below, this construction is applied to $\phi = -f(x, \cdot)$.

Persistent use across inner problems. Repeatedly initializing SCAR would repeat its curvature calibration. Our implementation instead retains the same $(\nu, \mathcal{M})$ across all calls to SCARHALF and SCAR-TARGET, even when ϕ changes. Inner iterates may be warm-started, but accelerated momentum and the internal AR stages are restarted at each AR call. The following consequence of the AR guarantee justifies this cross-objective reuse. Its proof is in [Appendix B](#); the sequential guarantee is not attributed to the original SCAR theorem. In particular, a successful halving does not certify $\nu \leq \mu$; the proof uses only the implication that a failed halving requires $\nu > \mu$.

Lemma 2.2 (Persistent SCAR halving). *Let $\{\phi_j\}$ be any sequence of differentiable functions on $\mathbb{R}^m$, all μ -strongly convex and all with L -Lipschitz gradients. Initialize a shared pair $(\nu, \mathcal{M})$ with $\mu \leq \nu_0 \leq L$ and $0 < \mathcal{M}_0 \leq 4L$.*

Use SCARHALF for each halving call, retaining $(\nu, \mathcal{M})$ between calls. Thus each attempt starts from the call's fixed u , updates $\mathcal{M}$ through AR, and either halves the fixed initial residual or divides ν by four and retries from u .

Every such call terminates and halves its initial residual. Across any N calls, including calls on different objectives or from unrelated initial points, the number of failed curvature guesses is at most

$$\left\lceil \log_4 \frac{\nu_0}{\mu} \right\rceil \leq \lceil \log_4(L/\mu) \rceil. \quad (2.22)$$

The total number of gradient evaluations, including the initial residual queries, is

$$O\left(\sqrt{\frac{L}{\mu}} \left[N + \left\lceil \log_4 \frac{\nu_0}{\mu} \right\rceil\right]\right). \quad (2.23)$$

The retained estimate satisfies $0 < \mathcal{M} \leq 4L$ throughout and $\mathcal{M} \leq 2L$ after the first nontrivial AR call.

2.3 The NC–SC setting

The minimax objective is

$$\min_{x \in \mathbb{R}^{d_x}} P(x), \quad P(x) := \max_{y \in \mathbb{R}^{d_y}} f(x, y). \quad (2.24)$$

We impose the following global regularity assumptions, as in Luo et al. [1].

Assumption 2.3 (NC–SC regularity). *The payoff f is twice continuously differentiable and satisfies:*

- (i) ∇f is ℓ -Lipschitz on $\mathbb{R}^{d_x} \times \mathbb{R}^{d_y}$;
- (ii) $\nabla^2 f$ is ρ -Lipschitz on $\mathbb{R}^{d_x} \times \mathbb{R}^{d_y}$ for some $\rho > 0$;
- (iii) $f(x, \cdot)$ is μ -strongly concave for every x , where $0 < \mu \leq \ell$;
- (iv) $P^* := \inf_x P(x) > -\infty$.

We write $\kappa := \ell/\mu$.

Strong concavity makes $y^*(x) := \arg \max_y f(x, y)$ unique and guarantees that $\nabla_{yy}^2 f(x, y)$ is nonsingular. Define the Schur complement

$$\mathcal{H}(x, y) := \nabla_{xx}^2 f(x, y) - \nabla_{xy}^2 f(x, y) (\nabla_{yy}^2 f(x, y))^{-1} \nabla_{yx}^2 f(x, y). \quad (2.25)$$

Lemma 2.3 (Value-function geometry). *Under [Assumption 2.3](#), $y^*(\cdot)$ is κ -Lipschitz, P is twice continuously differentiable, and*

$$\nabla P(x) = \nabla_x f(x, y^*(x)), \quad (2.26a)$$

$$\nabla^2 P(x) = \mathcal{H}(x, y^*(x)). \quad (2.26b)$$

Moreover, ∇P is $(1 + \kappa)\ell$ -Lipschitz and

$$\|\nabla^2 P(x) - \nabla^2 P(x')\| \leq M_P \|x - x'\|, \quad M_P := 4\sqrt{2}\kappa^3\rho. \quad (2.27)$$

These properties follow from Luo et al. [[1](#), Lemmas 1–3].

The residual gives the distance estimate

$$\|y - y^*(x)\| \leq \frac{\|\nabla_y f(x, y)\|}{\mu} \leq \kappa \|y - y^*(x)\|. \quad (2.28)$$

The first inequality follows from strong concavity and Cauchy–Schwarz; the second follows from ℓ -smoothness. This estimate is used in the analysis, not as an executable certificate when μ is unknown.

For the inner objective $q_x(y) := -f(x, y)$, the secant in [\(2.20\)](#) becomes

$$a_x := \frac{\|\nabla_y f(x, z) - \nabla_y f(x, y^{\text{in}})\|}{\|z - y^{\text{in}}\|}, \quad z \neq y^{\text{in}}. \quad (2.29)$$

By [\(2.21\)](#), $\mu \leq a_x \leq \ell$. It initializes persistent SCAR and scales the validation tolerance without evaluating either unknown curvature parameter.

3 A parameter-free minimax algorithm

We instantiate [Algorithm 1](#) and [Theorem 2.1](#) with the value function $F = P$, using persistent SCAR to realize the working and validation oracles. There are two distinct tasks: maintaining a sufficiently accurate working oracle along outer steps, and validating a possible stationary point. The first uses a step-dependent number of residual halvings; the second requests a smaller residual only when the outer step is interior and the corrected gradient is already small.

3.1 Corrected value-function oracles

At an approximate maximizer y , define the Newton-corrected value

$$\hat{P}(x, y) := f(x, y) - \frac{1}{2} \nabla_y f(x, y)^T (\nabla_{yy}^2 f(x, y))^{-1} \nabla_y f(x, y). \quad (3.1)$$

Since $\nabla_{yy}^2 f \prec 0$, the correction raises the raw payoff and is exact for a concave quadratic inner problem. The corrected gradient and Hessian estimators are

$$\hat{g}(x, y) := \nabla_x f(x, y) - \nabla_{xy}^2 f(x, y) (\nabla_{yy}^2 f(x, y))^{-1} \nabla_y f(x, y), \quad (3.2a)$$

$$\hat{H}(x, y) := \mathcal{H}(x, y). \quad (3.2b)$$

Unlike the raw plug-in gradient used in Luo et al. [[1](#), Algorithm 2], [\(3.2a\)](#) cancels the linear error in the approximate maximizer. The Hessian estimator remains the Schur complement.

Lemma 3.1 (Corrected value and derivative errors). *Under [Assumption 2.3](#), for every x and y ,*

$$\|\widehat{g}(x, y) - \nabla P(x)\| \leq \frac{(1 + \kappa)\rho}{2} \|y - y^*(x)\|^2, \quad (3.3a)$$

$$\|\widehat{H}(x, y) - \nabla^2 P(x)\| \leq (1 + \kappa)^2 \rho \|y - y^*(x)\|. \quad (3.3b)$$

If $\|y - y^*(x)\| \leq \mu/\rho$, then

$$|\widehat{P}(x, y) - P(x)| \leq \frac{7\rho}{24} \|y - y^*(x)\|^3. \quad (3.4)$$

The proof is given in [Section 4.1](#). The corrected value need not lie on a particular side of $P(x)$. Consequently, the acceptance analysis must account for the error of the stored proxy, rather than treat it as a certified function value.

3.2 Working accuracy and residual tracking

For $0 < \epsilon < 1$, set

$$\mathcal{A}_\epsilon := \log(e/\epsilon), \quad \sigma_0 := \mathcal{A}_\epsilon^{-1}, \quad c_0 := 2^{-12}, \quad r_\epsilon(\sigma) := \frac{\sqrt{\epsilon}}{4\sigma}. \quad (3.5)$$

The quantity $r_\epsilon(\sigma)$ sets the inner tracking accuracy and equals the radius at the gradient floor; the outer radius may be larger. A tuple $(\widehat{P}, \widehat{g}, \widehat{H})$ computed at (x, y) is called working-accurate at scale σ when

$$\|\nabla_y f(x, y)\| \leq c_0 \ell r_\epsilon(\sigma). \quad (3.6)$$

The unknown constant ℓ appears only in this analytical invariant, not in any test performed by the algorithm.

If the invariant holds before a step of length r , then

$$N_{\text{tr}}(r, \sigma) := \left\lceil \log_2 \left(1 + \frac{r}{c_0 r_\epsilon(\sigma)} \right) \right\rceil \quad (3.7)$$

successful persistent SCAR halvings at the candidate outer point restore it. One additional halving at an unchanged outer point restores it when σ is doubled. These statements follow from joint gradient Lipschitz continuity; the proof is in [Lemma 4.1](#). In particular, the number of required halvings is computed without ℓ or μ . Initialization uses the observable secant $a_0 \leq \ell$ to establish the invariant.

3.3 SCAR realization of the oracle interface

The working tuple is $(\widehat{P}, \widehat{g}, \widehat{H})$ from (3.1)–(3.2), together with its inner point y and cached residual gradient. The general UTR model (2.8) uses this tuple without modification. The next algorithm specifies the three working-query operations and the validator; the outer steps, acceptance tests, and scale updates are exactly those of [Algorithm 1](#). Every “persistent halving” below is one call to [SCARHALF](#). Initialization and validation use [SCARTARGET](#) with their displayed residual targets, whereas trial and refresh queries prescribe a number of [SCARHALF](#) calls. All these calls use the same shared calibration.

Algorithm 2: Parameter-free minimax UTR with persistent SCAR

- Input:** x_0, y_{-1} and $0 < \epsilon < 1$
- 1 Set $\sigma_0 = 1/\log(e/\epsilon)$ and $c_0 = 2^{-12}$
 - 2 Choose $z_0 \neq y_{-1}$, compute a_0 by (2.29), and initialize the shared SCAR calibration $\nu_0 = \mathcal{M}_0 = a_0$
 - 3 Define the working oracle $\mathcal{W}$ as follows:
 - 4 Initialization at x_0 : starting from y_{-1} , apply persistent halvings until $\|\nabla_y f(x_0, y)\| \leq c_0 a_0 r_\epsilon(\sigma_0)$; return the corrected tuple and its inner point y
 - 5 Trial at $x + d$ with scale σ : starting from the stored y , apply $N_{\text{tr}}(\|d\|, \sigma)$ successful persistent halvings to q_{x+d} ; return the corrected tuple and candidate y^+
 - 6 Refresh at unchanged x after σ doubles: starting from the old stored y , apply one successful persistent halving to q_x ; return the refreshed corrected tuple and inner point
 - 7 Define the validation oracle $\mathcal{V}$ at a candidate x^+ :
 - 8 Starting from its working inner point y^+ , choose $z^+ \neq y^+$ and compute a secant a^+ by (2.29)
 - 9 Keep a^+ fixed and continue persistent halvings until $\|\nabla_y f(x^+, y_v)\| \leq c_0 a^+ \epsilon^{3/2}$; return $(\hat{g}(x^+, y_v), \hat{H}(x^+, y_v))$
 - 10 Run Algorithm 1 with $F = P$, $(x_0, \epsilon, \sigma_0)$, and these oracles $\mathcal{W}, \mathcal{V}$; return its output
-

All inner calls share the calibration $(\nu, \mathcal{M})$, including calls on rejected candidates and failed validators. Accelerated momentum is restarted when the inner objective changes. Rejecting a candidate discards its point and corrected tuple but retains the updated calibration; a refresh starts from the old stored inner point, not the discarded candidate or validator. Accepted tuples are stored without further refinement. A zero-length trial needs no halving, whereas a scale-doubling refresh still requires one. Residual gradients are cached and reused, and every residual and secant query is included in the inner-gradient count.

3.4 Complexity guarantee

Theorem 3.1 (Class-uniform gradient-scaled complexity). *Fix $\ell \geq \mu > 0$ and $\rho > 0$. There exists $\epsilon_0 = \epsilon_0(\ell, \mu, \rho) \in (0, 1)$ such that the following holds for every twice continuously differentiable payoff $f : \mathbb{R}^{d_x} \times \mathbb{R}^{d_y} \rightarrow \mathbb{R}$ with ℓ -Lipschitz joint gradient, ρ -Lipschitz joint Hessian, μ -strongly concave inner sections, and a bounded-below value function $P(x) = \max_y f(x, y)$, and for every initialization (x_0, y_{-1}) . Set $\kappa = \ell/\mu$, $M_P = 4\sqrt{2}\kappa^3\rho$, and $\Delta_P = P(x_0) - \inf_x P(x)$. For every $0 < \epsilon \leq \epsilon_0$, run Algorithm 2 with exact evaluations of the derivatives of f used in its corrected oracles and globally solved trust-region subproblems. It terminates without using $\ell, \mu, \rho, \kappa, M_P$, or Δ_P as an input. Its output satisfies*

$$\|\nabla P(x_{\text{out}})\| < \epsilon, \quad \lambda_{\min}(\nabla^2 P(x_{\text{out}})) \geq -\frac{85}{16\sqrt{6}}\sqrt{M_P\epsilon}. \quad (3.8)$$

If Q counts all solved trust-region subproblems and N_y counts all inner-gradient evaluations, including the secant evaluations, then

$$Q = O\left(\Delta_P \sqrt{M_P} \epsilon^{-3/2}\right) + O_{\text{fixed}}(\log^3(e/\epsilon)), \quad (3.9a)$$

$$N_y = O\left(\Delta_P \sqrt{\kappa M_P} \epsilon^{-3/2}\right) + O_{\text{fixed}}(\sqrt{\kappa} \log^3(e/\epsilon)). \quad (3.9b)$$

Here O_{fixed} may depend on the fixed payoff, regularity constants, and initialization, but not on ϵ . The leading outer and inner terms are equivalently

$$O(\Delta_P \kappa^{3/2} \sqrt{\rho} \epsilon^{-3/2}) \quad \text{and} \quad O(\Delta_P \kappa^2 \sqrt{\rho} \epsilon^{-3/2}).$$

Using internal tolerance $(1536/7225)\epsilon$ returns an ϵ -SOSP of P , with $M = M_P$ in Definition 2.1, whenever that internal tolerance is at most ϵ_0 .

Remark 3.1 (Meaning of parameter freedom). *The same $\epsilon_0(\ell, \mu, \rho)$ works for the entire parameter class and every initialization. The constants in the additive O_{fixed} terms need not be uniform over that class. The algorithm neither knows this threshold nor certifies that $\epsilon \leq \epsilon_0$. Thus the theorem is a class-uniform asymptotic guarantee, not an observable certificate for every input accuracy. Sufficient inequalities defining such a threshold are given in (4.6). The residual-only obstruction and its relation to the secant-scaled validator are discussed in Section 5.*

Role of the general UTR theorem. The corrected working and validation tuples satisfy Assumption 2.2 below a class-uniform accuracy threshold. Thus the outer guarantee follows directly from Theorem 2.1; no separate minimax descent or scale-search argument is needed. Its cubic-path estimate pays for tracking on accepted steps, while its stored-value bound controls tracking at failed trials. Persistent SCAR shares one curvature calibration across the entire run. The remaining work is the initialization, scale-refresh, and candidate-validation cost. Section 4 gives all details.

4 NC–SC convergence analysis

The outer complexity and path budget are already available from Theorem 2.1. We prove the corrected-oracle estimates, verify its hypotheses for the SCAR construction, and then count the inner work.

4.1 Corrected value-function oracles

Proof of Lemma 3.1. Corrected value. Fix x and write $C = \nabla_{yy}^2 f(x, y)$, $q = \nabla_y f(x, y)$, and $\delta = y - y^*(x)$. Taylor expansion about y gives

$$v := q - C\delta, \quad \|v\| \leq \frac{\rho}{2}\|\delta\|^2, \quad P(x) = f(x, y) - q^T \delta + \frac{1}{2}\delta^T C \delta + R, \quad |R| \leq \frac{\rho}{6}\|\delta\|^3.$$

Using $q = C\delta + v$ and the definition of $\hat{P}$ yields

$$P(x) - \hat{P}(x, y) = R + \frac{1}{2}v^T C^{-1}v.$$

Strong concavity implies $\|C^{-1}\| \leq 1/\mu$. If $\|\delta\| \leq \mu/\rho$, then

$$\frac{1}{2}|v^T C^{-1}v| \leq \frac{\rho^2}{8\mu}\|\delta\|^4 \leq \frac{\rho}{8}\|\delta\|^3.$$

Adding the remainder proves (3.4).

Corrected gradient. Let

$$y_t := y^*(x) + t(y - y^*(x)), \quad t \in [0, 1].$$

Since f is twice continuously differentiable, the fundamental theorem of calculus applied to the two partial gradients along this segment gives the exact identities

$$\begin{aligned} \nabla_x f(x, y) - \nabla_x f(x, y^*(x)) &= \int_0^1 \nabla_{xy}^2 f(x, y_t)(y - y^*(x)) dt, \\ \nabla_y f(x, y) &= \int_0^1 \nabla_{yy}^2 f(x, y_t)(y - y^*(x)) dt, \end{aligned}$$

where the second identity uses $\nabla_y f(x, y^*(x)) = 0$. Substituting these identities into (3.2a) and using (2.26a), we obtain

$$\begin{aligned}\widehat{g}(x, y) - \nabla P(x) &= \int_0^1 \left[\nabla_{xy}^2 f(x, y_t) - \nabla_{xy}^2 f(x, y) \right. \\ &\quad \left. + \nabla_{xy}^2 f(x, y) (\nabla_{yy}^2 f(x, y))^{-1} (\nabla_{yy}^2 f(x, y) - \nabla_{yy}^2 f(x, y_t)) \right] \\ &\quad \times (y - y^*(x)) dt.\end{aligned}$$

This identity displays the cancellation of the linear inner error. Hessian Lipschitzness, together with

$$\left\| \nabla_{xy}^2 f(x, y) (\nabla_{yy}^2 f(x, y))^{-1} \right\| \leq \kappa,$$

bounds the norm of the integrand by

$$(1 + \kappa)\rho(1 - t)\|y - y^*(x)\|^2.$$

Integrating over $t \in [0, 1]$ proves (3.3a).

Corrected Hessian. Retain $C = \nabla_{yy}^2 f(x, y)$ and set

$$B := \nabla_{xy}^2 f(x, y), \quad B_* := \nabla_{xy}^2 f(x, y^*(x)), \quad C_* := \nabla_{yy}^2 f(x, y^*(x)).$$

The inverse identity gives

$$\begin{aligned}\left\| (\nabla_{yy}^2 f(x, y))^{-1} - (\nabla_{yy}^2 f(x, y^*(x)))^{-1} \right\| \\ \leq \frac{\rho}{\mu^2} \|y - y^*(x)\|.\end{aligned}$$

The Schur-complement products satisfy the exact telescoping identity

$$\begin{aligned}BC^{-1}B^T - B_*C_*^{-1}B_*^T &= (B - B_*)C^{-1}B^T + B_*(C^{-1} - C_*^{-1})B^T \\ &\quad + B_*C_*^{-1}(B - B_*)^T.\end{aligned}$$

Hessian Lipschitzness, $\|B\|, \|B_*\| \leq \ell$, and $\|C^{-1}\|, \|C_*^{-1}\| \leq 1/\mu$ bound these three terms by $\kappa\rho\|y - y^*(x)\|$, $\kappa^2\rho\|y - y^*(x)\|$, and $\kappa\rho\|y - y^*(x)\|$, respectively. Adding the xx -block error gives

$$\begin{aligned}\|\widehat{H}(x, y) - \nabla^2 P(x)\| &\leq \rho\|y - y^*(x)\| + 2\frac{\ell}{\mu}\rho\|y - y^*(x)\| \\ &\quad + \frac{\ell^2}{\mu^2}\rho\|y - y^*(x)\| \\ &= (1 + \kappa)^2\rho\|y - y^*(x)\|,\end{aligned}$$

which proves (3.3b). □

4.2 Tracking and oracle accuracy

Lemma 4.1 (Variable-step tracking). *Under Assumption 2.3, suppose (3.6) holds at (x, y) . For any step d of length r , applying $N_{\text{tr}}(r, \sigma)$ successful persistent SCAR halvings to q_{x+d} , starting from y , produces a working-accurate tuple at $x + d$ with the same scale. At an unchanged x , one successful halving restores the invariant after σ is doubled.*

Proof of Lemma 4.1. Lipschitz continuity of the full gradient gives

$$\|\nabla_y f(x + d, y)\| \leq \|\nabla_y f(x, y)\| + \ell r \leq \ell(c_0 r_\epsilon(\sigma) + r).$$

After $N_{\text{tr}}(r, \sigma)$ successful halvings, this is at most $c_0 \ell r_\epsilon(\sigma)$ by (3.7). The case $r = 0$ requires no halving. At an unchanged x , use $r_\epsilon(2\sigma) = r_\epsilon(\sigma)/2$. □

The error constants used only in the analysis are

$$C_g := \frac{c_0^2(1+\kappa)\rho\kappa^2}{32}, \quad C_v := \frac{7c_0^3\rho\kappa^3}{1536}. \quad (4.1)$$

We now verify the assumptions of the general UTR theorem.

Lemma 4.2 (Working and validation oracle accuracy). *Fix common constants (ℓ, μ, ρ) in [Assumption 2.3](#). For all sufficiently small $\epsilon > 0$, with a threshold depending only on these constants, every working-accurate tuple at $\sigma \geq \sigma_0 = \mathcal{A}_\epsilon^{-1}$ satisfies*

$$\|\hat{g} - \nabla P(x)\| \leq C_g \frac{\epsilon}{\sigma^2}, \quad |\hat{P} - P(x)| \leq C_v \frac{\epsilon^{3/2}}{\sigma^3}. \quad (4.2)$$

If also $\sigma^2 \geq M_P/6$, then

$$\begin{aligned} \|\hat{g} - \nabla P(x)\| &\leq \epsilon/1024, \\ \|\hat{H} - \nabla^2 P(x)\| &\leq \sigma\sqrt{\epsilon}/64, \\ |\hat{P} - P(x)| &\leq \epsilon^{3/2}/(4096\sigma). \end{aligned} \quad (4.3)$$

Furthermore, for any secant a_x from (2.29), any point y_v satisfying

$$\|\nabla_y f(x, y_v)\| \leq c_0 a_x \epsilon^{3/2} \quad (4.4)$$

obeys, at every $\sigma \geq \sigma_0$,

$$\begin{aligned} \|\hat{g}(x, y_v) - \nabla P(x)\| &\leq \epsilon/1024, \\ \|\hat{H}(x, y_v) - \nabla^2 P(x)\| &\leq \sigma\sqrt{\epsilon}/32. \end{aligned} \quad (4.5)$$

Consequently [Assumption 2.2](#) holds with $F = P$, $M = M_P$, and the displayed C_g, C_v . All conclusions hold uniformly over the parameter class, the outer and inner points, and the choices of secants.

Proof of Lemma 4.2. It is sufficient to restrict $\epsilon \in (0, 1)$ so that

$$\begin{aligned} \sqrt{\epsilon} \mathcal{A}_\epsilon &\leq \frac{4\mu}{c_0\kappa\rho}, & \epsilon^2 &\leq \frac{1}{512(1+\kappa)\rho c_0^2\kappa^2}, \\ \epsilon \mathcal{A}_\epsilon &\leq \frac{1}{32(1+\kappa)^2\rho c_0\kappa}, & \mathcal{A}_\epsilon &> \sqrt{\frac{6}{M_P}}. \end{aligned} \quad (4.6)$$

The first three left-hand sides tend to zero, and the last tends to infinity, as $\epsilon \downarrow 0$. Hence a positive threshold, depending only on (ℓ, μ, ρ) , makes these inequalities valid for all smaller positive ϵ . The last condition ensures $\sigma_0 < \sqrt{M_P/6}$, which will also be used in the termination proof.

Strong concavity and the working residual invariant give

$$e := \|y - y^*(x)\| \leq \frac{c_0\kappa\sqrt{\epsilon}}{4\sigma} \leq \frac{c_0\kappa}{4}\sqrt{\epsilon}\mathcal{A}_\epsilon \leq \frac{\mu}{\rho}.$$

The corrected-oracle bounds in [Lemma 3.1](#) yield

$$\|\hat{g} - \nabla P(x)\| \leq \frac{(1+\kappa)\rho}{2}e^2, \quad |\hat{P} - P(x)| \leq \frac{7\rho}{24}e^3.$$

Substituting the first bound on e proves (4.2).

At a safe scale $\sigma^2 \geq M_P/6$, use $M_P = 4\sqrt{2}\kappa^3\rho$ and $\kappa \geq 1$ to obtain

$$\begin{aligned} \|\hat{g} - \nabla P(x)\| &\leq \frac{3c_0^2(1+\kappa)}{64\sqrt{2}\kappa}\epsilon \leq \frac{3c_0^2}{32\sqrt{2}}\epsilon, \\ \|\hat{H} - \nabla^2 P(x)\| &\leq \frac{3c_0(1+\kappa)^2}{8\sqrt{2}\kappa^2}\sigma\sqrt{\epsilon} \leq \frac{3c_0}{2\sqrt{2}}\sigma\sqrt{\epsilon}, \\ |\hat{P} - P(x)| &\leq \frac{7c_0^3}{1024\sqrt{2}}\frac{\epsilon^{3/2}}{\sigma}. \end{aligned}$$

Since $c_0 = 2^{-12}$, these imply (4.3).

For validation, the secant bound $a_x \leq \ell$ and (4.4) give

$$\|y_v - y^*(x)\| \leq c_0 \kappa \epsilon^{3/2}.$$

The global derivative bounds in Lemma 3.1 then imply

$$\|\widehat{g}(x, y_v) - \nabla P(x)\| \leq \frac{(1 + \kappa) \rho c_0^2 \kappa^2}{2} \epsilon^3 \leq \frac{\epsilon}{1024},$$

by the second condition in (4.6). Similarly, its third condition and $\sigma \geq \mathcal{A}_\epsilon^{-1}$ give

$$\|\widehat{H}(x, y_v) - \nabla^2 P(x)\| \leq (1 + \kappa)^2 \rho c_0 \kappa \epsilon^{3/2} \leq \frac{\sqrt{\epsilon}}{32 \mathcal{A}_\epsilon} \leq \frac{\sigma \sqrt{\epsilon}}{32}.$$

This proves (4.5). None of the restrictions uses the payoff, initialization, evaluation points, or chosen secants. $\square$

4.3 Direct application and total inner work

Proof of Theorem 3.1. Choose $\epsilon_0 \in (0, 1)$ so that (4.6) holds for every $0 < \epsilon \leq \epsilon_0$. By Lemma 4.2, the corrected working tuples and the SCAR validator satisfy Assumption 2.2 with $F = P$, $M = M_P$, and the constants C_g, C_v in (4.1). Initialization and Lemma 4.1 maintain the working residual invariant. Every inner sequence terminates by Lemma 2.2 and positivity of the requested residual targets. Thus Theorem 2.1 applies directly to Algorithm 2. The onset restrictions depend only on the common constants (ℓ, μ, ρ) , and they also ensure $\sigma_0 = \mathcal{A}_\epsilon^{-1} < \sqrt{M_P/6}$. Consequently the scale cap in Theorem 2.1 gives $\bar{\sigma} = 2\sqrt{M_P/6}$.

We first specialize the budget B_ϵ in (2.11). Since $\sigma_0 = \mathcal{A}_\epsilon^{-1}$, its value-error term is $(23/7)C_v \epsilon^{3/2} \mathcal{A}_\epsilon^3$. At the initial stored tuple, (4.2) gives

$$\|\widehat{g}_0\| \leq \|\nabla P(x_0)\| + C_g \epsilon \mathcal{A}_\epsilon^2.$$

For each fixed payoff and initialization, this implies

$$\lceil \log_2(h_0/\epsilon) \rceil = O_{\text{fixed}}(\mathcal{A}_\epsilon), \quad 1 + \log_2 \left(1 + \frac{5C_g}{4\sigma_0^2} \right) = O_{\text{fixed}}(1 + \log \mathcal{A}_\epsilon),$$

where $h_0 = \max\{\|\widehat{g}_0\|, \epsilon\}$. Because $\epsilon^{3/2}/\sigma_0 = \epsilon^{3/2} \mathcal{A}_\epsilon$, substitution into (2.11) yields

$$B_\epsilon = \Delta_P + O_{\text{fixed}}(\epsilon^{3/2} \mathcal{A}_\epsilon^3). \quad (4.7)$$

The count bound (2.13), together with $\bar{\sigma} = 2\sqrt{M_P/6}$, now proves (3.9a). The output guarantee (2.12) gives (3.8) with the same scale bound. The tolerance conversion follows from $(85/(16\sqrt{6}))^2 = 7225/1536$.

It remains to count successful halvings. On an accepted step, set $z_k = \sigma_k r_k / \sqrt{\epsilon}$. Since $c_0 = 2^{-12}$,

$$N_{\text{tr},k} = \lceil \log_2(1 + 2^{14} z_k) \rceil \leq 16 + z_k^3.$$

For $0 \leq z \leq 1$, the ceiling is at most 15. For $z \geq 1$, use $1 + 2^{14} z \leq 2^{15} z$ and $\log_2 z \leq z^3$. Thus (2.13) and (2.14) give

$$\begin{aligned} \sum_{\text{accepted}} N_{\text{tr},k} &\leq 16K + \epsilon^{-3/2} \sum_{\text{accepted}} \sigma_k^3 r_k^3 \\ &\leq 16K + 8\bar{\sigma} B_\epsilon \epsilon^{-3/2} \\ &\leq 16392\bar{\sigma} B_\epsilon \epsilon^{-3/2}. \end{aligned} \quad (4.8)$$

To bound tracking at failed trials, put $L_P = (1 + \kappa)\ell$. Smoothness and the lower bound P^* imply

$$\|\nabla P(x)\|^2 \leq 2L_P(P(x) - P^*).$$

At every stored iterate, (2.14) and (4.2) therefore give

$$h \leq \bar{h}_\epsilon := \max \left\{ \epsilon, \sqrt{2L_P B_\epsilon} + C_g \epsilon \mathcal{A}_\epsilon^2 \right\} = O_{\text{fixed}}(1).$$

Since $r \leq R(h, \sigma)$, each failed trial requires at most

$$N_{\text{fail}} := \left\lceil \log_2 \left(1 + \frac{1}{c_0} \sqrt{\frac{\bar{h}_\epsilon}{\epsilon}} \right) \right\rceil = O_{\text{fixed}}(\mathcal{A}_\epsilon)$$

working halvings. A terminal trial has $h = \epsilon$ and $r < r_\epsilon(\sigma)$, so its working tracking uses at most $\lceil \log_2(1 + 1/c_0) \rceil = 13$ halvings. Each failure uses one additional halving for the refresh.

Immediately before a validation, the residual is at most $c_0 \ell \sqrt{\epsilon}/(4\sigma)$. The validation target is at least $c_0 \mu \epsilon^{3/2}$, so the number of additional halvings is at most

$$N_{\text{val}} := \left\lceil \log_2 \max \left\{ 1, \frac{\kappa \mathcal{A}_\epsilon}{4\epsilon} \right\} \right\rceil = O_{\text{fixed}}(\mathcal{A}_\epsilon).$$

There are at most $J + 1$ validations. Initialization uses at most

$$N_{\text{init}} := \left\lceil \log_2 \max \left\{ 1, \frac{\|\nabla_y f(x_0, y_{-1})\|}{c_0 a_0 r_\epsilon(\sigma_0)} \right\} \right\rceil = O_{\text{fixed}}(\mathcal{A}_\epsilon)$$

halvings. Consequently the total number of successful halvings obeys

$$N_{\text{half}} \leq \sum_{\text{accepted}} N_{\text{tr},k} + J N_{\text{fail}} + J + 13 + (J + 1) N_{\text{val}} + N_{\text{init}}. \quad (4.9)$$

By (2.13), $J = O_{\text{fixed}}(1 + \log \mathcal{A}_\epsilon)$. Equations (4.7) and (4.8) therefore imply

$$N_{\text{half}} = O(\Delta_P \sqrt{M_P} \epsilon^{-3/2}) + O_{\text{fixed}}(\mathcal{A}_\epsilon^3).$$

All inner objectives $q_x = -f(x, \cdot)$ share the same (μ, ℓ) . Apply Lemma 2.2 once to the complete sequence, including rejected candidates, refreshes, and validators. The initial and validation secants require at most $2(J + 2)$ additional gradient evaluations, giving

$$N_y = O(\sqrt{\kappa} [N_{\text{half}} + \lceil \log_4 \kappa \rceil] + J + 1). \quad (4.10)$$

This proves (3.9b). Finally, $\sqrt{M_P} = \Theta(\kappa^{3/2} \sqrt{\rho})$ and $\sqrt{\kappa M_P} = \Theta(\kappa^2 \sqrt{\rho})$ give the stated leading dependence. $\square$

5 Scope of the parameter-free guarantee

This section elaborates on Remark 3.1. The distinction is between an algorithm that needs no problem parameters and a finite-accuracy certificate that can itself be verified without those parameters.

5.1 The residual-only obstruction

A parameter-free inner method such as SCAR can return a point satisfying the prescribed and directly verifiable condition $\|\nabla_y f(x, y)\| \leq \tau$. This fact alone does not provide a certificate for the value or gradient of $P(x) = \max_y f(x, y)$ that is uniform over problem classes having no positive common lower bound on the strong-concavity modulus. The issue is not merely a loss in the complexity bound: an outer stopping test based only on a residual-corrected oracle can be false.

The following scalar family shows that this difficulty persists even for the corrected value (3.1). Let

$$c := \frac{1}{2}, \quad \psi(t) := \frac{t^2}{2} + c(1 - \cos t), \quad A := \frac{1 - c}{2\pi^2},$$

and, for $0 < \mu \leq 1/4$, define

$$f_\mu(x, y) := A \tanh(x/A) (1 - \cos(2\pi\sqrt{\mu}y)) - \psi(2\sqrt{\mu}y - 1). \quad (5.1)$$

Because $1 - c \leq \psi''(t) \leq 1 + c$,

$$\partial_{yy}^2 f_\mu(x, y) = 4A\pi^2\mu \tanh(x/A) \cos(2\pi\sqrt{\mu}y) - 4\mu\psi''(2\sqrt{\mu}y - 1) \leq -2(1 - c)\mu = -\mu.$$

Thus $f_\mu(x, \cdot)$ is globally μ -strongly concave. Its second derivatives are uniformly bounded for $0 < \mu \leq 1/4$. Its third derivatives are also uniformly bounded: the four distinct block types have bounds $12/A^2$, $2\pi/A$, π^2 , and $A\pi^3 + c$, respectively. Hence the joint gradient-smoothness and Hessian-Lipschitz constants admit finite upper bounds independent of μ . Moreover,

$$P_\mu(x) \geq f_\mu\left(x, \frac{1}{2\sqrt{\mu}}\right) = 2A \tanh(x/A) \geq -2A,$$

so the value function is bounded below. At $x = 0$ and $y = 1/(2\sqrt{\mu})$, $-\partial_{yy}^2 f_\mu(x, y) = 4\mu\psi''(0) = 6\mu$. Consequently, the largest valid global strong-concavity modulus lies between μ and 6μ and genuinely tends to zero along the family.

At $x = 0$, the exact maximizer is $y_\mu^*(0) = 1/(2\sqrt{\mu})$. At the point $y = 0$, however,

$$|\partial_y f_\mu(0, 0)| = 2\sqrt{\mu} |\psi'(-1)| = (2 + \sin 1)\sqrt{\mu} \longrightarrow 0, \quad \widehat{g}_\mu(0, 0) = 0, \quad \widehat{H}_\mu(0, 0) = 0.$$

In contrast, Danskin's theorem gives

$$P'_\mu(0) = \partial_x f_\mu\left(0, \frac{1}{2\sqrt{\mu}}\right) = 1 - \cos \pi = 2.$$

The corrected value is wrong by a fixed amount as well. Namely, $P_\mu(0) = 0$, whereas

$$\widehat{P}_\mu(0, 0) = -\psi(-1) + \frac{\psi'(-1)^2}{2\psi''(-1)} \approx 0.0647, \quad (5.2)$$

independently of μ . Therefore a residual tending to zero does not uniformly control either the corrected value or the corrected gradient over the union of these parameter classes. The smoothness constants remain bounded while the best strong-concavity modulus tends to zero.

Consequently, choosing SCAR as a residual module removes the inner solver's dependence on ℓ and μ , but the residual alone does not yield a single certificate valid across all positive strong-concavity moduli. Nor does a prescribed finite sequence of positive tolerances repair the issue. For any finite list $\tau_0, \dots, \tau_J > 0$, choose

$$0 < \mu < \min \left\{ \frac{1}{4}, \left(\frac{\min_j \tau_j}{2 + \sin 1} \right)^2 \right\}$$

and repeat each inner call at the outer point $x = 0$, initialized at $y = 0$. Every residual test is then already satisfied and may return the same false outer derivatives. This observation covers any predetermined finite number of tolerance-halving phases. It does not claim that indefinite refinement cannot eventually move the inner iterate; it shows that a finite prescribed refinement depth is not a certificate.

Once a positive common lower bound μ is fixed, the condition

$$\tau < (2 + \sin 1)\sqrt{\mu}$$

excludes $y = 0$ as a legal return of an inner call with residual target τ . More generally, every legal return satisfies $\|y - y^*(x)\| \leq \tau/\mu$. Thus the family illustrates why a residual-to-oracle estimate must retain its dependence on the common strong-concavity bound. It does not rule out eventual accuracy for any fixed problem.

This example concerns prescribed absolute residual targets. It does not establish failure of [Algorithm 2](#), whose working and validation targets use observable secants and whose accepted inner points are tracked persistently. [Theorem 3.1](#) asserts uniformity after (ℓ, μ, ρ) have been fixed; it makes no claim that one threshold works over their union as $\mu \downarrow 0$.

5.2 Validation and stored-oracle bookkeeping

SCAR returns a verified residual, not a certified lower bound on μ or an interval containing $P(x)$. Its line-search estimate satisfies the tested inequalities but is not a certified global smoothness bound. A successful residual halving likewise does not certify that its current strong-convexity guess lies below the true modulus. In the minimax proof, the common regularity constants are used to establish the working and validation error bounds, even though they are absent from the executable algorithm.

The corrected value $\hat{P}$ is neither a guaranteed upper nor a guaranteed lower bound on P . An accepted trial at an unsafe scale need not decrease P . The proof instead telescopes the stored potential used in the proof of [Theorem 2.1](#) and pays for every positive jump caused by a scale refresh; see [Lemma A.4](#). This requires three implementation rules: the scale is nondecreasing, an accepted corrected tuple is retained without extra refinement, and a rejected candidate is discarded before the current tuple is refreshed. Refining a stored inner iterate without accounting for the resulting proxy change would invalidate that argument.

The validator is used only after an interior trial at $h = \epsilon$. Its residual target $c_0 a_x \epsilon^{3/2}$ controls inner accuracy; it is not a trust-region scale or a Hessian threshold. Under the class-uniform small-accuracy conditions, its tests imply true second-order stationarity. They do not reveal whether those conditions hold. Refining or doubling the requested accuracy alone therefore does not provide the missing observable certificate. Such a certificate would require an additional argument or information, for example a certified inner gap, a value-function interval, a known strong-concavity lower bound, or problem-specific structure.

6 Conclusion

The parameter-free minimax method is a direct oracle realization of an inexact UTR theorem for general unconstrained optimization. The theorem separates the construction of approximate values and derivatives from outer progress and provides the accepted path and stored-value bounds needed for inner-work analysis. Corrected envelope oracles and persistent SCAR satisfy its hypotheses in the NC–SC setting, yielding leading complexities $O(\Delta_P \kappa^{3/2} \sqrt{\rho} \epsilon^{-3/2})$ and $O(\Delta_P \kappa^2 \sqrt{\rho} \epsilon^{-3/2})$ for outer subproblems and inner-gradient evaluations, respectively.

The guarantee holds uniformly over each class with fixed common regularity constants once ϵ is sufficiently small. The algorithm does not certify that it has entered this regime. An observable all-accuracy certificate and a treatment of inexact trust-region solves remain beyond the present analysis.

A Proof of the inexact UTR theorem

All results in this appendix concern the general unconstrained objective F and the oracle interface of [Assumption 2.2](#). No minimax structure is used.

A.1 A common trust-region estimate

Lemma A.1 (Common inexact trust-region step estimate). *Suppose [Assumption 2.1](#) holds. At a point x , let g and a symmetric H satisfy*

$$\|g - \nabla F(x)\| \leq \delta_0, \quad \|H - \nabla^2 F(x)\| \leq \xi.$$

For $a \geq 0$ and $R > 0$, let d be a global solution of

$$\min_{\|u\| \leq R} q(u) := g^T u + \frac{1}{2} u^T (H + aI) u,$$

let λ be the multiplier supplied by (2.3), and write $r = \|d\|$. Let a candidate gradient g^+ satisfy $\|g^+ - \nabla F(x + d)\| \leq \delta_+$. Then

$$q(d) \leq -\frac{\lambda}{2} r^2, \tag{A.1a}$$

$$F(x) - F(x + d) \geq \frac{a - \xi}{2} r^2 + \frac{\lambda}{2} r^2 - \frac{M}{6} r^3 - \delta_0 r, \tag{A.1b}$$

$$\|g^+\| \leq (a + \xi + \lambda) r + \frac{M}{2} r^2 + \delta_0 + \delta_+. \tag{A.1c}$$

Proof. Apply (2.3) with $B = H + aI$. Its stationarity and positive-semidefiniteness conditions give

$$q(d) = -\frac{1}{2} d^T (H + aI) d - \lambda r^2 \leq -\frac{\lambda}{2} r^2,$$

which proves (A.1a). The Taylor bound and the current endpoint errors imply

$$F(x) - F(x + d) \geq -g^T d - \frac{1}{2} d^T H d - \delta_0 r - \frac{\xi}{2} r^2 - \frac{M}{6} r^3.$$

Since $-g^T d - d^T H d / 2 = -q(d) + a r^2 / 2$, the model bound yields (A.1b).

For the candidate gradient, decompose the two endpoint errors, Taylor remainder, Hessian error, and model residual:

$$\begin{aligned} g^+ &= [g^+ - \nabla F(x + d)] + [\nabla F(x + d) - \nabla F(x) - \nabla^2 F(x) d] \\ &\quad + [\nabla F(x) - g] + [\nabla^2 F(x) - H] d + [g + H d]. \end{aligned}$$

The stationarity part of (2.3), again with $B = H + aI$, gives $g + H d = -(a + \lambda) d$. Taking norms proves (A.1c). $\square$

A.2 Safe-scale acceptance and validation

Lemma A.2 (Safe scale for inexact UTR). *Under [Assumptions 2.1](#) and [2.2](#), the following hold for every generated trial of [Algorithm 1](#). If $\sigma^2 \geq M/6$, every ordinary trial of [Algorithm 1](#) satisfies both acceptance tests, and every trial with $h = \epsilon$ and $r < R(h, \sigma)$ passes validation. At any scale $\sigma \geq \sigma_0$, successful validation implies*

$$\|\nabla F(x^+)\| < \epsilon, \quad \lambda_{\min}(\nabla^2 F(x^+)) \geq -\frac{85}{32} \sigma \sqrt{\epsilon}. \tag{A.2}$$

Proof of Lemma A.2. Apply Lemma A.1 to F with

$$a = \sigma\sqrt{h} + \frac{\sigma\sqrt{\epsilon}}{64}, \quad \xi = \frac{\sigma\sqrt{\epsilon}}{64}, \quad \delta_0 = \delta_+ = \frac{\epsilon}{1024}.$$

The endpoint errors follow from (2.5). Since $r \leq \sqrt{h}/(4\sigma)$ and $M \leq 6\sigma^2$,

$$\frac{M}{6}r^3 \leq \frac{W}{4}.$$

Thus (A.1b) gives

$$F(x) - F(x+d) \geq \frac{W}{4} + \frac{U}{2} - \frac{\epsilon r}{1024}.$$

Young's inequality and $h \geq \epsilon$ imply

$$\frac{\epsilon r}{1024} \leq \frac{W}{8} + \frac{2\epsilon^2}{1024^2\sigma\sqrt{h}} \leq \frac{W}{8} + \frac{2T}{1024^2}.$$

After paying the two working-value errors,

$$\widehat{F} - \widehat{F}^+ \geq \frac{W}{8} + \frac{U}{2} - \left(\frac{2}{1024^2} + \frac{2}{4096} \right) T \geq \frac{W}{8} + \frac{U}{4} - \frac{T}{1024}.$$

For the gradient test, (A.1c) gives

$$\|\widehat{g}^+\| \leq \lambda r + \frac{h}{4} + \frac{h}{128} + \frac{3h}{16} + \frac{h}{512} = \lambda r + \frac{229h}{512} \leq \lambda r + \frac{h}{2}.$$

Hence every ordinary trial is accepted at a safe scale.

If $h = \epsilon$ and $r < R(h, \sigma)$, complementarity gives $\lambda = 0$, and the model shift is $a = (65/64)\sigma\sqrt{\epsilon}$. Applying (A.1c) to the true endpoint gradient ($\delta_+ = 0$), with $\delta_0 = \epsilon/1024$ and $\xi = \sigma\sqrt{\epsilon}/64$, gives

$$\|\nabla F(x+d)\| \leq \left(\frac{66}{256} + \frac{3}{16} + \frac{1}{1024} \right) \epsilon = \frac{457}{1024} \epsilon.$$

Moreover, model positive semidefiniteness, the current Hessian error, and Hessian Lipschitz continuity imply

$$\begin{aligned} \lambda_{\min}(\nabla^2 F(x+d)) &\geq -a - \xi - Mr \\ &\geq -\left(\frac{66}{64} + \frac{3}{2} \right) \sigma\sqrt{\epsilon} = -\frac{81}{32} \sigma\sqrt{\epsilon}. \end{aligned}$$

The validation error bounds therefore give

$$\|g_v\| \leq \frac{458}{1024} \epsilon, \quad \lambda_{\min}(H_v) \geq -\frac{82}{32} \sigma\sqrt{\epsilon},$$

so validation succeeds. At any generated scale, a successful validation and (2.6) imply

$$\|\nabla F(x^+)\| \leq \frac{\epsilon}{2} + \frac{\epsilon}{1024} < \epsilon, \quad \lambda_{\min}(\nabla^2 F(x^+)) \geq -\left(\frac{21}{8} + \frac{1}{32} \right) \sigma\sqrt{\epsilon}.$$

This proves (A.2). □

A.3 Accepted-step and scale-refresh budgets

For $h \geq \epsilon$, define the gradient level and stored potential

$$\mathcal{L}_\epsilon(h) := \lceil \log_2(h/\epsilon) \rceil, \quad \Phi := \widehat{F} + \frac{\epsilon^{3/2}}{512\sigma} \mathcal{L}_\epsilon(h). \tag{A.3}$$

Lemma A.3 (Accepted-step potential decrease). *Every ordinary accepted step of Algorithm 1 satisfies*

$$\Phi - \Phi^+ \geq \frac{W}{32} + \frac{T}{1024} + \frac{U}{16}, \quad (\text{A.4})$$

where the scale is unchanged and Φ is defined by (A.3).

Proof. First note that if $h > \epsilon$ and $h^+ \leq \max\{h/2, \epsilon\}$, then

$$\mathcal{L}_\epsilon(h^+) \leq \mathcal{L}_\epsilon(h) - 1. \quad (\text{A.5})$$

For $\epsilon < h \leq 2\epsilon$, the two levels are 1 and 0. For $h > 2\epsilon$, the result follows from $\lceil t-1 \rceil = \lceil t \rceil - 1$. We will also use, for $c \geq 1$ and $h^+ \leq ch$,

$$\mathcal{L}_\epsilon(h^+) - \mathcal{L}_\epsilon(h) \leq \lceil \log_2 c \rceil \leq 1 + \log_2 c. \quad (\text{A.6})$$

Suppose first that the accepted step is interior. Then $\lambda = 0$ and $U = 0$. Since this is an ordinary trial, $h > \epsilon$. The gradient acceptance test implies $h^+ \leq \max\{h/2, \epsilon\}$, so (A.5) and the value test yield

$$\Phi - \Phi^+ \geq \frac{W}{8} - \frac{T}{1024} + \frac{T}{512} = \frac{W}{8} + \frac{T}{1024}.$$

This proves (A.4) in the interior case.

For a boundary step, put $z = \lambda r/h$. The gradient test gives $h^+/h \leq \max\{1, 1/2 + z\}$. By (A.6),

$$\mathcal{L}_\epsilon(h^+) - \mathcal{L}_\epsilon(h) \leq 1 + \log_2 \max\{1, 1/2 + z\} \leq 1 + \frac{z}{\ln 2}.$$

Since $r = \sqrt{h}/(4\sigma)$,

$$W = \frac{h^{3/2}}{16\sigma} \geq \frac{T}{16}, \quad U = \frac{zh^{3/2}}{4\sigma}, \quad Tz \leq 4U.$$

Consequently,

$$\begin{aligned} \Phi - \Phi^+ &\geq \frac{W}{8} + \frac{U}{4} - \frac{T}{1024} - \frac{T}{512} \left(1 + \frac{z}{\ln 2}\right) \\ &\geq \frac{W}{8} + \left(\frac{1}{4} - \frac{1}{128 \ln 2}\right) U - \frac{3T}{1024} \\ &\geq \frac{5W}{64} + \left(\frac{1}{4} - \frac{1}{128 \ln 2}\right) U. \end{aligned}$$

Finally, $5W/64 \geq W/32 + T/1024$ and $1/4 - 1/(128 \ln 2) \geq 1/16$, proving the claim. $\square$

To account for changes of scale, define

$$\begin{aligned} E(\sigma) &:= C_v \frac{\epsilon^{3/2}}{\sigma^3}, & E_0 &:= E(\sigma_0), & T_0 &:= \frac{\epsilon^{3/2}}{\sigma_0}, \\ h_0 &:= \max\{\|\widehat{g}_0\|, \epsilon\}, & \mathcal{L}_0 &:= \mathcal{L}_\epsilon(h_0), & D_0 &:= 1 + \log_2 \left(1 + \frac{5C_g}{4\sigma_0^2}\right). \end{aligned} \quad (\text{A.7})$$

Here $\widehat{g}_0$ is the working gradient after initialization. The budget in (2.11) can therefore be written as

$$B_\epsilon = \Delta_F + \frac{23}{7} E_0 + \frac{T_0}{512} (\mathcal{L}_0 + D_0).$$

Write $\sigma_* := \sqrt{M/6}$. Every failed trial doubles σ , and Lemma A.2 excludes failures at or above σ_* . If $\sigma_0 \geq \sigma_*$ there are no failures. Otherwise, at most $\lceil \log_2(\sigma_*/\sigma_0) \rceil$ doublings are possible. Consequently every finite prefix satisfies

$$J \leq \lceil \log_2 \max\{1, \sigma_*/\sigma_0\} \rceil, \quad \sigma \leq \bar{\sigma} := \max\{\sigma_0, 2\sigma_*\}. \quad (\text{A.8})$$

Lemma A.4 (Cumulative accepted-step budget). *Under the preceding conditions, every finite prefix satisfies*

$$\sum_{\text{accepted}} \left(\frac{W_k}{32} + \frac{T_k}{1024} + \frac{U_k}{16} \right) \leq B_\epsilon. \quad (\text{A.9})$$

If K denotes its number of accepted steps, then

$$K \leq 1024\bar{\sigma}B_\epsilon\epsilon^{-3/2}, \quad \sum_{\text{accepted}} \sigma_k^2 r_k^3 \leq 8B_\epsilon. \quad (\text{A.10})$$

Every stored iterate satisfies $F(x) - F^* \leq B_\epsilon$.

Proof. At a refresh, x is unchanged and the scale goes from σ to 2σ . By (2.4), the working-value jump is at most

$$E(\sigma) + E(2\sigma) = \frac{9}{8}E(\sigma).$$

The scales at failures form a doubling sequence, so the sum of all positive value jumps is at most

$$\frac{9}{8}E_0 \sum_{j \geq 0} 2^{-3j} = \frac{9}{7}E_0.$$

For the gradient-level term, the two working-gradient errors give

$$\|\hat{g}_{\text{new}} - \hat{g}_{\text{old}}\| \leq \frac{5C_g\epsilon}{4\sigma^2}.$$

The map $g \mapsto \max\{\|g\|, \epsilon\}$ is 1-Lipschitz and $h_{\text{old}} \geq \epsilon$. Hence

$$\frac{h_{\text{new}}}{h_{\text{old}}} \leq 1 + \frac{5C_g}{4\sigma^2}, \quad \mathcal{L}_\epsilon(h_{\text{new}}) - \mathcal{L}_\epsilon(h_{\text{old}}) \leq D_0.$$

Write $\mathcal{L}_{\text{old}}$ and $\mathcal{L}_{\text{new}}$ for these two levels. Since $T_{\text{new}} = T_{\text{old}}/2$ and the levels are nonnegative, the positive jump of the level contribution is at most

$$\left[\frac{T_{\text{old}}}{1024} (\mathcal{L}_{\text{new}} - 2\mathcal{L}_{\text{old}}) \right]_+ \leq \frac{T_{\text{old}}D_0}{1024}.$$

Its cumulative positive jump is therefore bounded by

$$\frac{T_0D_0}{1024} \sum_{j \geq 0} 2^{-j} = \frac{T_0D_0}{512}.$$

The initial potential is at most $F(x_0) + E_0 + T_0\mathcal{L}_0/512$, and every stored potential is at least $F^* - E_0$. Telescoping (A.4) and paying the two refresh bounds gives

$$\sum_{\text{accepted}} \left(\frac{W}{32} + \frac{T}{1024} + \frac{U}{16} \right) \leq \Delta_F + 2E_0 + \frac{9}{7}E_0 + \frac{T_0}{512}(\mathcal{L}_0 + D_0) = B_\epsilon.$$

The same upper bound on the stored potential, together with $F(x) \leq \Phi + E_0$, gives $F(x) - F^* \leq B_\epsilon$.

By (A.8), $T_k \geq \epsilon^{3/2}/\bar{\sigma}$, which proves the bound on K . The radius constraint also gives

$$4\sigma_k^2 r_k^3 \leq \sigma_k \sqrt{h_k} r_k^2 = W_k.$$

Combining this with (A.9) proves the cubic-path bound. □

A.4 Termination and the theorem’s conclusions

Proof of Theorem 2.1. Every oracle call is finite by the interface assumptions. By Lemma A.2, failed trials can occur only below $\sqrt{M/6}$, and the scale bound (A.8) limits their number uniformly over all finite prefixes. Lemma A.4 likewise bounds the number of accepted steps of every finite prefix by $1024\bar{\sigma}B_\epsilon\epsilon^{-3/2}$. Every nonterminal trial is either accepted or failed. Hence an infinite run would contradict one of these finite-prefix bounds, and the algorithm terminates.

At termination, there have been K accepted trials, J failed trials, and one successful validation trial, each solving one subproblem. Thus $Q = K + J + 1$, proving (2.13). Every validation either terminates or causes a failed trial, so there are at most $J + 1$ validation calls. The returned-point estimate (A.2) and $\sigma \leq \bar{\sigma}$ prove (2.12). Finally, Lemma A.4 proves (2.14) for every stored iterate and every finite prefix, as claimed. $\square$

B Persistent SCAR

For completeness, the secant bounds (2.21) follow from strong monotonicity and Cauchy–Schwarz:

$$\mu\|z - u\|^2 \leq \langle \nabla\phi(z) - \nabla\phi(u), z - u \rangle \leq \|\nabla\phi(z) - \nabla\phi(u)\| \|z - u\|.$$

Dividing by $\|z - u\|^2$ gives the lower bound; the upper bound is gradient Lipschitz continuity.

Proof of Lemma 2.2. Fix an attempted AR call on ϕ_j from u , and write $u_j^* = \arg \min \phi_j$ and $R = \|\nabla\phi_j(u)\|$. Strong convexity gives

$$\|u - u_j^*\| \leq \frac{\|\nabla\phi_j(u)\|}{\mu} = \frac{R}{\mu}.$$

The AR estimate (2.17) consequently yields

$$\|\nabla\phi_j(v)\| \leq \frac{\nu}{2}\|u - u_j^*\| \leq \frac{\nu}{2\mu}R.$$

Hence a halving attempt must succeed whenever $\nu \leq \mu$. In particular, failure implies $\nu > \mu$ and the update after a failure leaves $\nu > \mu/4$. Since the initial guess satisfies $\nu_0 \geq \mu$, throughout the complete sequence we have $\nu \geq \mu/4$. After $\lceil \log_4(\nu_0/\mu) \rceil$ failures, the guess is at most μ , and no later objective can cause a failure: the same strong-convexity bound applies to every ϕ_j . This proves (2.22) and finite termination of every halving call.

The initial line-search estimate satisfies $\mathcal{M}_0 \leq 4L$. Every nontrivial AR call returns $\mathcal{M}^+ \leq 2L$ by (2.17). All objectives have the same upper smoothness bound L , so this retained estimate remains admissible even when the next objective changes. Thus the AR guarantee applies inductively to every attempted call.

Each attempt uses $O(1 + \sqrt{L/\nu}) = O(\sqrt{L/\mu})$ gradient evaluations because $\nu \geq \mu/4$ and $L/\mu \geq 1$. There are at most N successful AR attempts and at most $\lceil \log_4(\nu_0/\mu) \rceil$ failed attempts. The at most N initial residual queries, including queries for calls with zero initial residual, fit in the same bound. Summing proves (2.23). Returning u when $R = 0$ already satisfies the required residual reduction and does not change either retained estimate. $\square$

References

- [1] Luo Luo, Yujun Li, and Cheng Chen. Finding second-order stationary points in nonconvex-strongly-concave minimax optimization. In *Advances in Neural Information Processing Systems*, volume 35, pages 36667–36679, 2022.

- [2] Ziyi Chen, Zhengyang Hu, Qunwei Li, Zhe Wang, and Yi Zhou. A cubic regularization approach for finding local minimax points in nonconvex minimax optimization. *Transactions on Machine Learning Research*, 2023. URL <https://openreview.net/forum?id=jVMMdg31De>.
- [3] Tongliang Yao and Zi Xu. Two trust region type algorithms for solving nonconvex-strongly concave minimax problems. *Scientia Sinica Mathematica*, 56(2):437–454, 2026. doi: 10.1360/SCM-2024-0063.
- [4] Junlin Wang and Zi Xu. Gradient norm regularization second-order algorithms for solving nonconvex-strongly concave minimax problems. *Journal of Scientific Computing*, 105(2):41, 2025. doi: 10.1007/s10915-025-03069-8.
- [5] Jia-Hao Chen, Zi Xu, and Hui-Ling Zhang. A homogeneous second-order descent ascent algorithm for nonconvex-strongly-concave minimax problems. *arXiv preprint arXiv:2602.14058*, 2026.
- [6] Bohao Ma, Nachuan Xiao, and Junyu Zhang. Line-search and adaptive step sizes for nonconvex-strongly-concave minimax optimization. *arXiv preprint arXiv:2601.10086*, 2026.
- [7] Bohao Ma, Nachuan Xiao, and Junyu Zhang. A single-loop regularized Newton method for nonconvex-strongly-concave minimax optimization. *arXiv preprint arXiv:2606.07003*, 2026.
- [8] Pierre Ablin, Gabriel Peyré, and Thomas Moreau. Super-efficiency of automatic differentiation for functions defined as a minimum. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 32–41. PMLR, 2020. URL <https://proceedings.mlr.press/v119/ablin20a.html>.
- [9] Guojun Zhang, Kaiwen Wu, Pascal Poupart, and Yaoliang Yu. Newton-type methods for minimax optimization. *arXiv preprint arXiv:2006.14592*, 2020. URL <https://arxiv.org/abs/2006.14592>.
- [10] Yuntian Jiang, Chang He, Chuwen Zhang, Dongdong Ge, Bo Jiang, and Yinyu Ye. Beyond nonconvexity: A universal trust-region method with new analyses. *Journal of Scientific Computing*, 2026. arXiv:2311.11489.
- [11] Guanghui Lan, Yuyuan Ouyang, and Zhe Zhang. Optimal and parameter-free gradient minimization methods for convex and nonconvex optimization. *Mathematical Programming*, 2026. doi: 10.1007/s10107-026-02352-2.
- [12] Maher Nouiehed, Maziar Sanjabi, Tianjian Huang, Jason D. Lee, and Meisam Razaviyayn. Solving a class of non-convex min-max games using iterative first order methods. In *Advances in Neural Information Processing Systems*, volume 32, pages 14934–14942, 2019.
- [13] Tianyi Lin, Chi Jin, and Michael I. Jordan. On gradient descent ascent for nonconvex-concave minimax problems. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 6083–6093. PMLR, 2020.
- [14] Tianyi Lin, Chi Jin, and Michael I. Jordan. Near-optimal algorithms for minimax optimization. In *Proceedings of the Thirty-Third Conference on Learning Theory*, volume 125 of *Proceedings of Machine Learning Research*, pages 2738–2779. PMLR, 2020.
- [15] Siqi Zhang, Junchi Yang, Cristóbal Guzmán, Negar Kiyavash, and Niao He. The complexity of nonconvex-strongly-concave minimax optimization. In *Proceedings of the Thirty-Seventh Conference on Uncertainty in Artificial Intelligence*, volume 161 of *Proceedings of Machine Learning Research*, pages 482–492. PMLR, 2021.

- [16] Ruichen Jiang, Ali Kavis, Qiujiang Jin, Sujay Sanghavi, and Aryan Mokhtari. Adaptive and optimal second-order optimistic methods for minimax optimization. In *Advances in Neural Information Processing Systems*, volume 37, pages 94130–94162, 2024.
- [17] Ruichen Jiang and Aryan Mokhtari. Generalized optimistic methods for convex-concave saddle point problems. *SIAM Journal on Optimization*, 35(3):2066–2097, 2025. doi: 10.1137/24M1630475.
- [18] Jun-Lin Wang, Zi Xu, and Hui-Ling Zhang. A fully parameter-free second-order algorithm for convex-concave minimax problems. *Journal of Machine Learning Research*, 27(76):1–32, 2026.
- [19] Yurii Nesterov. *Introductory Lectures on Convex Optimization: A Basic Course*, volume 87 of *Applied Optimization*. Kluwer Academic Publishers, Boston, 2004. doi: 10.1007/978-1-4419-8853-9.
- [20] David G. Luenberger and Yinyu Ye. *Linear and Nonlinear Programming*. Springer, Cham, 5 edition, 2021. doi: 10.1007/978-3-030-85450-8.